Lista de Exercícios – Programação Web – 2026.2

Aluno(a): Davi Roberto Pereira Barbosa Data: 10/11/2026

Instruções

A lista tem 28 questões: 17 de múltipla escolha, 10 de código e 1 desafio integrador.

- Nas questões de múltipla escolha, marque **apenas uma** alternativa com X.
- Nas questões de código, entregue arquivos `.html`, `.css` e `.js` separados, salvo indicação contrária. O código deve rodar diretamente no navegador, sem bibliotecas externas.

Parte 1 – Questões de múltipla escolha

1. Qual elemento semântico deve envolver o conteúdo principal e único de uma página?

( ) a) `<section>`

( ) b) `<article>`

( ) c) `<div id="main">`

(X) d) `<main>`

2. Qual é a principal função do atributo `alt` na tag `<img>`?

(X) a) Fornecer texto alternativo para leitores de tela e para quando a imagem não carrega

( ) b) Exibir um título quando o mouse passa sobre a imagem

( ) c) Definir a largura da imagem

( ) d) Transformar a imagem em link

3. Qual é a forma correta de associar um rótulo a um campo de formulário?

( ) a) `<label name="email">` e `<input id="email">`

(X) b) `<label for="email">` e `<input id="email">`

( ) c) `<label id="email">` e `<input for="email">`

( ) d) `<label class="email">` e `<input class="email">`

4. Qual valor do atributo `type` faz o navegador validar nativamente o formato de um e-mail?

( ) a) `text`

( ) b) `mail`

(X) c) `email`

( ) d) `address`

5. Qual dos elementos abaixo é exibido em linha (*inline*) por padrão?

( ) a) `<p>`

( ) b) `<div>`

(X) c) `<span>`

( ) d) `<ul>`

6. Sobre a declaração `<!DOCTYPE html>`, é correto afirmar que ela:

( ) a) É uma tag que precisa ser fechada no fim do documento

( ) b) Serve para importar o arquivo CSS

( ) c) É opcional e não altera a renderização

(X) d) Informa ao navegador que o documento é HTML5 e evita o modo de compatibilidade (*quirks mode*)

7. Qual dos seletores abaixo tem a **maior** especificidade?

( ) a) `.menu a`

( ) b) `nav ul li a`

( ) c) `a:hover`

(X) d) `#topo`

8. Considere o CSS abaixo. Qual é a largura total ocupada pela caixa na tela?

```
.caixa {
  box-sizing: border-box;
  width: 200px;
  padding: 20px;
  border: 5px solid #333;
}
```

(X) a) 200px

( ) b) 250px

( ) c) 240px

( ) d) 225px

9. Em um container com `display: flex` e `flex-direction: row`, qual declaração centraliza os itens na horizontal?

( ) a) `align-items: center`

(X) b) `justify-content: center`

( ) c) `text-align: center`

( ) d) `vertical-align: middle`

10. A unidade `rem` é relativa:

(X) a) Ao `font-size` do elemento raiz (`html`)

( ) b) Ao `font-size` do elemento pai

( ) c) À largura da viewport

( ) d) À altura da viewport

11. Qual é o resultado de `typeof null` em JavaScript?

( ) a) `"null"`

( ) b) `"undefined"`

( ) c) `"number"`

(X) d) `"object"`

12. Quais são os resultados de `"5" + 3` e `"5" - 3`, respectivamente?

(X) a) `“53”` e `2`

( ) b) `8` e `2`

( ) c) `“53”` e `“2”`

( ) d) `8` e `“53”`

13. Qual método retorna o **primeiro** elemento do documento que corresponde a um seletor CSS?

( ) a) `document.getElementsByClassName()`

( ) b) `document.querySelectorAll()`

(X) c) `document.querySelector()`

( ) d) `document.getElementById()`

14. O que o código abaixo exibe no console?

```
for (var i = 0; i < 3; i++) {
  setTimeout(function () {
    console.log(i);
  }, 0);
}
```

( ) a) `0 1 2`

(X) b) `3 3 3`

( ) c) `2 2 2`

( ) d) `undefined undefined undefined`

15. Sobre `let`, é correto afirmar que:

(X) a) Tem escopo de bloco e não pode ser redeclarada no mesmo escopo

( ) b) Tem escopo de função, assim como `var`

( ) c) Não pode ter seu valor reatribuído

( ) d) Só pode ser usada dentro de funções

16. O que acontece ao executar o código abaixo?

```
const aluno = { nota: 7 };
aluno.nota = 9;
console.log(aluno.nota);
```

( ) a) `TypeError`, pois `const` não pode ser alterada

(X) b) Exibe `9`, pois `const` impede reatribuir a variável, mas não alterar o objeto

( ) c) `SyntaxError`

( ) d) Exibe `7`, pois a alteração é ignorada

17. Qual é o resultado da expressão abaixo?

```
[1, 2, 3, 4].filter(n => n % 2 === 0).map(n => n * 10);
```

( ) a) `[10, 30]`

( ) b) `[2, 4]`

( ) c) `[10, 20, 30, 40]`

(X) d) `[20, 40]`

Parte 2 – Questões de código

18. Crie a página inicial de um blog com:

- `<html lang="pt-BR">`, `<meta charset>` e `<meta name="viewport">`
- `<header>` com o nome do blog e um `<nav>` com 3 links em lista
- `<main>` com 2 `<article>`, cada um com título, data em `<time datetime="...">` e um parágrafo
- `<aside>` com uma lista de "Posts populares" e `<footer>` com direitos autorais
- Nenhuma `<div>` pode ser usada onde existe um elemento semântico adequado

19. Crie um formulário com os campos abaixo, todos com `<label>` associado:

- Nome completo (obrigatório, mínimo de 3 caracteres)
- E-mail (obrigatório) e data de nascimento
- Curso em `<select>` com 3 opções e turno em `radio` (manhã, tarde, noite)
- Observações em `<textarea>` e uma `checkbox` "Aceito os termos" (obrigatória)
- Agrupe os dados pessoais em um `<fieldset>` com `<legend>`

20. Crie o CSS da página da questão 18:

- Defina as cores do tema como variáveis em `:root` (ex.: `--cor-primaria`) e use-as em todo o arquivo
- Deixe o `<nav>` horizontal com Flexbox e espaçamento com `gap`
- Links do menu sem sublinhado, com mudança de cor no `:hover` e `transition` de 0.3s

21. Crie 3 cards lado a lado, cada um com imagem, nome, preço e botão "Comprar".

- Container com `display: flex`, `flex-wrap: wrap` e `gap`
- Cards com `border-radius`, `box-shadow` e imagem com `object-fit: cover`
- No `:hover`, o card sobe 4px (`transform: translateY(-4px)`) com transição suave

22. Estilize o formulário da questão 19:

- Campos com `width: 100%` e `box-sizing: border-box`
- Estilo de `:focus` visível e personalizado (não remova o contorno sem substituí-lo)
- Borda vermelha para campos inválidos e verde para válidos, aplicada só depois que o usuário digitar (dica: `:user-invalid` ou `:invalid:not(:placeholder-shown)`)

> Nas questões 23 a 26, use apenas JavaScript clássico: `var`, `function`, laços `for`/`while` e concatenação com `+`.

23. Escreva a função `ehPrimo(n)`, que retorna `true` se `n` for primo e `false` caso contrário. Use-a para exibir no console todos os primos entre 1 e 100.

24. Crie uma página com um número (começa em 0) e três botões: "+", "−" e "Zerar".

- Use `addEventListener` e `textContent`, nunca `onclick` no HTML
- O valor não pode ficar negativo
- O número fica verde quando for maior que 10 e vermelho quando for 0

25. Valide com JavaScript o formulário da questão 19:

- Use `event.preventDefault()` para impedir o envio quando houver erros
- Erros: nome com menos de 3 caracteres ou termos não aceitos
- Mostre cada mensagem de erro em um elemento logo abaixo do campo, sem usar `alert()`
- Se não houver erros, exiba "Inscrição realizada com sucesso!" na página

26. Crie campos para peso (kg) e altura (m) e um botão "Calcular".

- Calcule IMC = peso ÷ altura² e exiba com 2 casas decimais (`toFixed`)
- Classifique: abaixo de 18,5 = abaixo do peso; 18,5 a 24,9 = normal; 25 a 29,9 = sobrepeso; 30 ou mais = obesidade
- Mostre uma mensagem de erro se algum campo estiver vazio ou com valor menor ou igual a zero

27. Use `let` / `const` e arrow functions, sem `var` e sem laços `for`. Com o array abaixo, resolva cada item com `map`, `filter` e `reduce`:

```
const produtos = [
  { nome: "Teclado", preco: 150, categoria: "periféricos", estoque: 10 },
  { nome: "Mouse", preco: 80, categoria: "periféricos", estoque: 0 },
  { nome: "Monitor", preco: 900, categoria: "telas", estoque: 4 },
  { nome: "Notebook", preco: 3500, categoria: "computadores", estoque: 2 },
  { nome: "Headset", preco: 220, categoria: "periféricos", estoque: 7 }
];
```

1. Gere um array só com os nomes em letras maiúsculas
2. Gere um array só com os produtos que têm estoque
3. Calcule o valor total em estoque (preço × estoque) com `reduce`
4. Agrupe os produtos por categoria em um objeto, com `reduce`
5. Exiba cada produto com template literal no formato `Teclado — R$ 150,00`, usando `toLocaleString("pt-BR", { style: "currency", currency: "BRL" })`

Parte 3 – Desafio integrador

28. Construa uma aplicação de lista de tarefas usando HTML, CSS e JavaScript ES6+, sem bibliotecas.

Requisitos

- ☐ HTML semântico: `<header>`, `<main>`, formulário com `<label>` e lista em `<ul>`
- ☐ Adicionar tarefa pelo formulário, sem recarregar a página e sem aceitar texto vazio
- ☐ Marcar e desmarcar tarefa como concluída (texto riscado com `text-decoration`)
- ☐ Remover tarefa
- ☐ Filtros “Todas”, “Pendentes” e “Concluídas”
- ☐ Contador de tarefas pendentes, atualizado a cada ação
- ☐ Layout responsivo com Flexbox e cores definidas em variáveis CSS